Submitting your project

For this project, you must hand in:

- gomoku.py

You may resubmit any time before the deadline. Your last submission will be graded for correctness and will be the version that the TA sees. Submissions must be done using MarkUs.

Main block

All your testing code should be inside the `if __name__ == "__main__"` block. You must **not** use global variables in this project.

Clarifications and discussion board

Important clarifications and/or corrections to the project, should there be any, will be posted on the ESC180 Piazza. You are responsible for monitoring the announcements there.

How you will be marked

Your work will be marked for correctness using MarkUs and for your understanding during the project demo.

Autotesting for Correctness

Your functions need to work as described below. We will run your functions using a Python 3 interpreter. Please ensure that you are running Python 3 as well. To check what version of Python you are running, you can run the following in your Python shell:

```
import sys
sys.version
```

To mark your code for correctness, you will need to upload your code to the MarkUs platform. Since the code is graded automatically, it is critical that the code does not contain syntax errors, as this will cause your mark to be 0. If your file contains syntax errors, you will see that on MarkUs right away. In addition, your functions must be implemented precisely as described. Their signature must exactly match the handout and their behaviour must be exactly as specified. This means that the functions should not print anything unless the project specifications require that and that they should return exactly what the project handout describes.

Some of the tests used to evaluate your project can be run by you. However, the tests we provide you with are not exhaustive. We will test your code with more test cases for evaluation. As a result, you should test your code yourself to be confident it works.

We encourage you to upload your code to MarkUs early to ensure you know how to do so, potentially even before completing all of the functions. Note, MarkUs is currently set to allow you to run tests on your code 12 times per hour.

Project Demo

You will be assigned a 15-minute assessment time on **Wednesday, November 11**, during your assigned lab period. You will demo your project to a TA, who will ask you questions about your code and may ask you to write/modify some code on the spot. Although you are working in pairs, you will receive an individual mark based on your individual performance during the project demo. You will be expected to arrive at least 10 minutes before your scheduled project demo time to set up your project code. Once you have completed your demo, you are welcome to leave.

Note, you will not be assessed on documentation or code style for this project. However, having clear comments could help you navigate your own code and answer questions more easily.

Documentation

Note that you will not be assessed on documentation or code style for this project. However, having clear comments could help you navigate your own code and answer questions more easily. The following are some instances of when you should add comments to your code:

- For every function, as a docstring, to describe the parameters of the function and what the function does. See below for more details on docstrings.
- Before every global variable declaration, to describe what kind of information the variable stores and what properties (if any) that information is supposed to have throughout the execution of the code.
- Before all complicated sections of code, to help the reader understand what that code section is trying to do.
- In general, comments should not simply restate what the code does (this does not add any useful information to the code). Comments should add information that is implicit in the code, e.g., about what purpose a computation serves, or why a certain section of code is written the way it is.

Guidelines for writing docstrings

- Describe precisely *what* the function does.
- Do not reveal *how* the function does it.
- Make the purpose of every parameter clear.

- Refer to every parameter by name.
- Be clear about whether the function returns a value, and if so, what.
- Explain any conditions that the function assumes are true. Examples: “n is an int”, “n != 0”, “the height and width of p are both even”.
- Be concise.
- Ensure that the text you write is grammatically correct.
- Write the docstring as a command (e.g., “Return the first...”) rather than a statement (e.g., “Returns the first...”).

Style

Good style practices should be adhered to when writing your code. This includes the following:

- Use Python style conventions for your function and variable names. In particular, please use “pothole case”: lowercase letters with words separated by underscores (_), to improve readability.
- Choose good names for your functions and variables. For example, `num_coffee_cups` is more helpful and readable than `ncc`.
- To make sure your program will be formatted correctly is never to mix spaces and tabs when indenting —use only tabs, or only spaces.
- Put a blank space before and after every operator. For example:

```
b = 3 > x and 4 - 5 < 32 # good style: easy to read
b= 3>x and 4-5<32 # bad style: hard to read
```

- Write a docstring comment for each function. (See above for guidelines on the content of your docstrings.) Put a blank line after every docstring comment.
- Each line must be less than 80 characters long, including tabs and spaces. You should break up long lines using `\`.
- Your code should be readable and readily understandable.

The Gomoku Game

In this assignment you will implement a simple (and imperfect) AI engine for the game Gomoku, played on an 8 × 8 board. In Gomoku, there are two players. One player plays with black stones, and the other player plays with white stones. A player moves by placing a stone on an empty square on the board. The player who plays with black stones always moves first. After the first move, the players alternate. A player wins if they have placed five of their stones in a sequence, either horizontally, or vertically, or diagonally. Please read http://en.wikipedia.org/wiki/Gomoku for more information about Gomoku. We will be playing the standard variant.

The Starter Code

The file `gomoku.py` contains the starter code for the Gomoku game and AI engine. The program works as follows. The computer (which plays with the black stones) always moves first. After the first move, the user's and the computer's moves alternate. The computer determines its move by finding the move that maximises the return value of the `score` function (which is provided).

Functions in `gomoku.py` accept `board` as one of their arguments. This is the representation of the Gomoku board. The square `(y,x)` on the board is stored in `board[y][x]`. The value of the square is:

- `" "`, if the square is empty,
- `"b"`, if the square has a black stone on it, and
- `"w"`, if the square has a white stone on it.

See the function `print_board()` for an example of using `board`.

An important part of the Gomoku AI engine is finding contiguous sequences of stones of the same colour on the Gomoku board. There are four possible directions for a sequence: left-to-right, top-to-bottom, upper-left-to-lower-right, and upper-right-to-lower-left. Note that we do not consider, for example, the direction right-to-left, since a right-to-left sequence can be represented as a left-to-right sequence. The direction of a sequence can be represented by a pair of numbers `(d_y, d_x)` as follows:

- `(0,1)`: direction left-to-right. For example, the sequence of stones of the same colour on coordinates `(5,2), (5,3), (5,4), (5,5)` is a sequence in direction left-to-right. Note that we say that the last stone in the sequence is at location `(5,5)`, not at location `(5,2)`.
- `(1,0)`: direction top-to-bottom. For example, a sequence of stones of the same colour on coordinates `(3,1), (4,1), (5,1)` is a top-to-bottom sequence. Note that we say that the last stone in the sequence is at location `(5,1)`, not at location `(3,1)`.

- `(1,1)`: direction upper-left-to-lower-right. For example, a sequence of stones of the same colour on coordinates `(2,3)`, `(3,4)`, `(4,5)` is an upper-left-to-lower-right sequence. Note that we say that the last stone in the sequence is at location `(4,5)`, not at location `(2,3)`.
- `(1,-1)`: direction upper-right-to-lower-left. For example, a sequence of stones of the same colour on coordinates `(5,5)`, `(6,4)`, `(7,3)` is an upper-right-to-lower-left sequence. Note that we say that the last stone in the sequence is at location `(7,3)`, not at location `(5,5)`.

A sequence can be:

- *open*: a stone can be put on a square at either side of the sequence.
- *closed*: the sequence is blocked on both sides, so that no stone can be placed on either side of the sequence. This can occur either because the sequence begins/ends near the border of the board, or because there is a stone of a different colour in the location immediately next to the beginning/end of the sequence.
- *semi-open*: the sequence is neither open nor closed.

Here are some examples of sequences and their classifications.

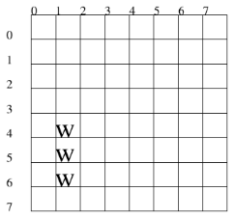


Figure 1: (1,0)

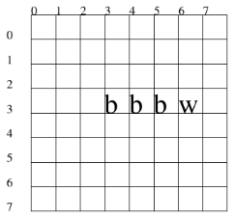


Figure 2: (0,1)

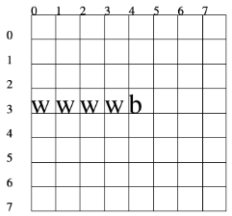


Figure 3: (0,1)

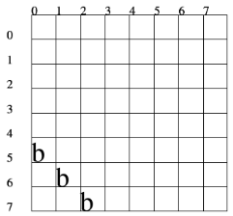


Figure 4: (1,1)

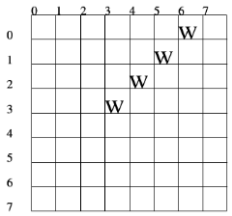


Figure 5: (1,-1)

- Figure 1: An open sequence with 3 white stones. The direction is `(1,0)`. The last stone is at location `(6,1)`.
- Figure 2: A semi-open sequence with 3 black stones. The direction is `(0,1)`. The last stone is at location `(3,5)`.
- Figure 3: A closed sequence with 4 white stones. The direction is `(0,1)`. The last stone is at location `(3,3)`.
- Figure 4: A closed sequence with 3 black stones. The direction is `(1,1)`. The last stone is at location `(7,2)`.
- Figure 5: A semi-open sequence with 4 white stones. The direction is `(1,-1)`. The last stone is at location `(3,3)`.

The file `gomoku.py` contains implementations of the following functions:

- `print_board(board)`
  This function prints out the Gomoku board.
- `score(board)`
  This function computes and returns the score for the position of the board. It assumes that black has just moved.
- `play_gomoku(board_size)`
  This function allows the user to play against a computer on a board of size `board_size` × `board_size`. This function interacts with the AI engine by calling the function `search_max()`, which you will write.
- `put_seq_on_board(board, y, x, d_y, d_x, length, col)`
  This helper function adds the sequence of stones of colour `col` of length `length` to `board`, starting at location `(y,x)` and moving in the direction `(d_y, d_x)`. This function facilitates the testing of the AI engine.
- `analysis(board)`
  This function analyses the position of the board by computing the number of open and semi-open sequences of both colours.

Do not modify these functions!

Functions to Implement

Write an AI engine for the Gomoku game by implementing the following functions.

Part (a) **`is_empty(board)`**

This function returns `True` iff there are no stones on the board `board`.

Part (b) **`is_bounded(board, y_end, x_end, length, d_y, d_x)`**

This function analyses the sequence of length `length` that ends at location `(y_end, x_end)`. The function returns `"OPEN"` if the sequence is open, `"SEMIOPEN"` if the sequence if semi-open, and `"CLOSED"` if the sequence is closed.

Assume that the sequence is complete (i.e., you are not just given a subsequence) and valid, and contains stones of only one colour.

Part (c) **`detect_row(board, col, y_start, x_start, length, d_y, d_x)`**

This function analyses the row (let's call it **R**) of squares that starts at the location `(y_start,x_start)` and goes in the direction `(d_y,d_x)`. Note that this use of the word *row* is different from "a *row* in a table". Here the word *row* means a sequence of squares, which are adjacent either horizontally, or vertically, or diagonally. The function returns a tuple whose first element is the number of open sequences of colour `col` of length `length` in the row **R**,

and whose second element is the number of semi-open sequences of colour `col` of length `length` in the row **R**.

Assume that `(y_start,x_start)` is located on the edge of the board. Only *complete* sequences count. For example, column 1 in Fig. 1 is considered to contain one open row of length 3, and no other rows.

Assume `length` is an integer greater or equal to 2.

Part (d) `detect_rows(board, col, length)`

This function analyses the board `board`. The function returns a tuple, whose first element is the number of open sequences of colour `col` of length `length` **on the entire board**, and whose second element is the number of semi-open sequences of colour `col` of length `length` **on the entire board**.

Only *complete* sequences count. For example, Fig. 1 is considered to contain one open row of length 3, and no other rows.

Assume `length` is an integer greater or equal to 2.

Part (e) `search_max(board)`

This function uses the function `score()` (provided) to find the optimal move for black. It finds the location `(y,x)`, such that `(y,x)` is empty and putting a black stone on `(y,x)` maximizes the score of the board as calculated by `score()`. The function returns a tuple `(y, x)` such that putting a black stone in coordinates `(y, x)` maximizes the potential score (if there are several such tuples, you can return any one of them). After the function returns, the contents of `board` **must** remain the same.

Part (f) `is_win(board)`

This function determines the current status of the game, and returns one of `["White won", "Black won", "Draw", "Continue playing"]`, depending on the current status on the board. The only situation where `"Draw"` is returned is when `board` is full.

Testing Your Code

Test your code thoroughly, and make sure that all your functions work as required by the spec. While you do not earn points directly for thoroughly testing your code, doing that is the only way to make sure your functions work.